

Логи DSP

Контекст DSP (Demand-Side Platform) - это платформа для рекламодателей, которая позволяет закупать рекламу на множестве сайтов и в приложениях из одного интерфейса. Примеры: Yandex Direct, Google Display, Criteo

DSP получает сырые логи событий:

- `impression`
- `click`
- `conversion`
- `bid_request`

События приходят не по порядку, часть дублируется, идентификаторы — `device_id/user_id/cookie`.

Бизнес-цель

Бизнес-цель - построить datamart для аналитиков/рекламодателей, которая: атрибутирует конверсии, отсекает фрод, выдает бизнес-метрики по кампаниям: `spend`, `valid_clicks`, `attributed_conversions`, `post_view_conversions`.

Бизнес-трудности - события приходят не по порядку, бывают дубликаты, часть полей может быть пустой, объем огромный, исчисляется терабайтами. Данные должны быть верными, есть SLA на перерасчет.

User-stories:

Как менеджер по перформансу, я хочу видеть конверсии, атрибутированные к моим кампаниям по правилу last-touch, чтобы корректно оценивать CPA/ROAS и не "переплачивать" за post-view там, где был реальный клик.

Как антифрод-аналитик, я хочу, чтобы pipeline применял последовательные правила к событиям устройства, чтобы отбрасывать подозрительные клики: без предшествующего показа, слишком быстрые, burst-паттерны.

Как специалист по оптимизации, я хочу получать метрики `spend` и `valid_clicks` по `campaign_id`, `hour`, чтобы автоматические биддеры/пейсинг алгоритмы опирались на "чистые" клики и не разгоняли ставки на ботах.

Почему здесь RDD выигрывает у DataFrame

1) Компактнее (меньше кода)

В RDD это естественно выражается как:

- распарсить бинарь → `Event`
- `keyBy(device_id)`
- `repartitionAndSortWithinPartitions` по `(device_id, ts)`

- один проход `mapPartitions`: внутри итератора держим небольшой state и выдаём уже “очищенные/атрибутированные” записи

В DataFrame обычно получится “лес” из:

- UDF/deserialize
- окон `Window.partitionBy(device_id).orderBy(ts)`
- `lag/lead`, `rangeBetween`
- self-join’ы или explode-структуры для “найти последний валидный клик”
- сложности с дедупом “с допуском” без тяжёлых конструкций

2) Быстрее (ручные оптимизации)

RDD позволяет явно сделать оптимизации, которые DataFrame не всегда делает/делает хуже:

- **repartitionAndSortWithinPartitions**: один shuffle и дальше линейный скан
- **map-side combine** там, где возможно (например, для агрегаций spend/clicks)
- в `mapPartitions`:
 - переиспользовать объекты, парсеры protobuf, буферы (минимум аллокаций/GC)
 - обрезать state (TTL) без материализации окон
- контроль партиционирования: можно бить по `device_id` с “солением” hot-ключей (skew), если есть суперактивные устройства/боты

3) Надёжнее (управление памятью)

Здесь риск OOM в DataFrame возникает из-за:

- окон, которые могут держать большие фреймы
- sort + window + join → большие буферы/спиллы, иногда непредсказуемо

В RDD можно:

- хранить промежуточное в **SER** (`persist(MEMORY_AND_DISK_SER)`)
- ограничивать state в `mapPartitions` (например, держать только последние N событий или последние 7 дней по compact-структуре)
- аккуратно работать итератором “потокowo”, **не собирая массивы событий на device_id**

4) Гибче

Ключевой момент: это задача не “SQL-агрегации”, а **последовательностный алгоритм** (state machine) по отсортированным событиям.

RDD подходит идеально, потому что:

- вы пишете детерминированный алгоритм “скан ленты событий” (как в streaming), но на батче
- легко добавлять сложные правила (burst, зависимость impression↔click, эвристики)
- можно внедрить компактные структуры (LRU, счетчики, bloom filter на дедуп, скетчи), не вываливаясь в тяжелые UDF/joins

Ниже — **мелкие парные примеры** “одна и та же часть одной задачи”, где видно, что для *последовательной логики по device_id* DataFrame выглядит тяжело, а RDD — естественно и

компактно.

Пример на конкретной задаче:

Лента событий по `device_id` (impression/click/conversion).

Нужно: **valid_click** (зависит от предыдущих событий) и **атрибуция conversion** (last-touch 7d иначе post-view 24h), дальше агрегаты по `campaign_id`, `hour`.

1) "Сделать ленту событий по `device_id` во времени"

DataFrame: плохо: окна/сортировки становятся фундаментом всего

```
from pyspark.sql import Window as W, functions as F
w = W.partitionBy("device_id").orderBy("ts_ms")
events = events.withColumn("prev_type", F.lag("event_type").over(w))
```

Проблема: как только логика "смотрит назад/вперёд", вы почти неизбежно попадаете в **Window** (а дальше их становится много).

RDD: хорошо: вы сразу получаете "stream-подобный" порядок

```
sorted_rdd = (events_rdd
    .map(lambda e: ((e.device_id, e.ts_ms), e))
    .repartitionAndSortWithinPartitions(2000)
    .values())
```

Дальше вся логика — один `mapPartitions` линейным сканом.

2) Правило антифрода: "клик валиден, если был impression на тот же `ad_id` за 30 минут"

DataFrame: плохо: `rangeBetween` окно

```
w30 = (W.partitionBy("device_id", "ad_id")
    .orderBy("ts_ms")
    .rangeBetween(-30*60*1000, 0))

imp_ts = F.max(F.when(F.col("event_type")=="impression",
    F.col("ts_ms"))).over(w30)
events = events.withColumn("has_imp_30m", imp_ts.isNotNull())
```

RDD (хорошо: 2 строки state)

```
# внутри mapPartitions, события уже отсортированы
if et == "impression": last_imp_by_ad[ad] = ts
```

```
if et == "click": is_valid = (ad in last_imp_by_ad) and (ts -  
last_imp_by_ad[ad] <= MIN30)
```

3) Правило антифрода: "too-fast" click

DataFrame: плохо: снова зависит от окна/агрегации в окне

```
events = events.withColumn(  
    "too_fast",  
    (F.col("event_type")=="click") & (F.col("ts_ms") - imp_ts < 50)  
)
```

RDD: хорошо: то же условие без инфраструктуры окон

```
if et == "click" and ad in last_imp_by_ad:  
    too_fast = (ts - last_imp_by_ad[ad]) < 50
```

4) Правило антифрода: burst ">20 кликов за 10 минут и 0 конверсий"

DataFrame: плохо: ещё одно rangeBetween + 2 суммирования

```
w10 = (W.partitionBy("device_id")  
        .orderBy("ts_ms")  
        .rangeBetween(-10*60*1000, 0))  
  
clicks10 = F.sum(F.expr("event_type='click'")).over(w10)  
convs10 = F.sum(F.expr("event_type='conversion'")).over(w10)  
events = events.withColumn("burst_bad", (clicks10 > 20) & (convs10 == 0))
```

RDD: хорошо: явный контроль окна и памяти

```
trim(click_ts, ts, MIN10); trim(conv_ts, ts, MIN10)  
if et == "click": click_ts.append(ts)  
if et == "conversion": conv_ts.append(ts)  
burst_bad = (len(click_ts) > 20) and (len(conv_ts) == 0)
```

5) Атрибуция: last-touch click за 7 дней, иначе post-view impression за 24 часа

DataFrame: плохо: ещё два длинных окна 7d и 24h

```

w7d =
W.partitionBy("device_id","campaign_id").orderBy("ts_ms").rangeBetween(-7*
DAY, 0)
w24h =
W.partitionBy("device_id","campaign_id").orderBy("ts_ms").rangeBetween(-
DAY, 0)

last_click = F.last(F.when(F.col("valid_click"), F.col("ts_ms")),
True).over(w7d)
last_imp = F.last(F.when(F.col("event_type")=="impression",
F.col("ts_ms")), True).over(w24h)
attr_ts = F.coalesce(last_click, last_imp)

```

RDD: хорошо: последнее подходящее событие — это просто state

```

if et == "click" and is_valid: last_valid_click_by_camp[camp] = ts
if et == "impression":      last_imp_by_camp[camp] = ts
if et == "conversion":
    attr_ts = last_valid_click_by_camp.get(camp) or
last_imp_by_camp.get(camp)

```

6) Главный удар в пользу RDD: контроль памяти на skew'e

DataFrame: плохо: окно не "сбрасывается" на hot device

```

# window-логика не даёт вам места, где вы явно "сбросили state" для
device_id
# (фактически фреймы/буферы считаются движком)

```

RDD (хорошо: 1 понятный if — и состояние гарантированно ограничено)

```

if dev != cur_dev:
    cur_dev = dev
    last_imp_by_ad.clear(); last_imp_by_camp.clear()
    click_ts.clear(); conv_ts.clear()

```